A1-12 sbrk – step by step

every line of code, and the theory behind it

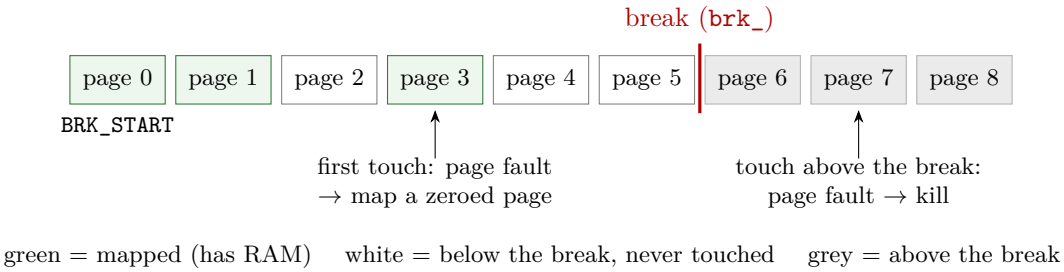
How to use this. Do one step at a time: read it, type the code yourself (do not copy-paste – typing is how it sticks), check the "Done when" box, then go on. Before you start: `task start a1-12`. At the end: `task test`, done when **7 of 7** checks pass and the last access kills the process.

Theory: `concepts.pdf` §3 sbrk & demand paging, §4 page faults, §16 page tables. The same pattern as A0-03 heaplimit – only the number you store and the syscall differ.

The goal (level 1)

```
Name:         sbrk (POSIX, unistd.h - the stub exists)
Nr:           1700
Description:  Every process has a program break, starting at 0x600000000000.
              sbrk(increment) moves it by increment bytes (may be negative) and returns
              the OLD break. Memory below the break is usable; it is mapped (zeroed) on
              first access by the page fault handler. Shrinking unmaps every page that is
              now completely above the break. An access above the break kills the process.
Return Value: old break, or (void*) -1 if the new break would be below the start or more
              than 4096 pages above it
```

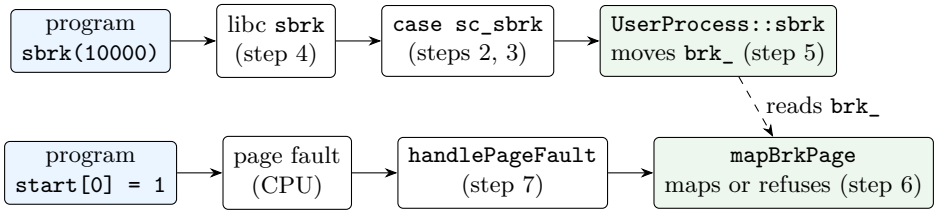
sbrk = "set break". The **break** is the end of the heap: everything from the heap start up to the break may be used, everything above it may not.



Pages are **not** mapped left to right: the program touches them in any order, and each one gets RAM at its first touch. All threads of a process share this one heap and this one break.

The order: 7 steps

#	What	File	Why in this order
1	constants + the break variable	`UserProcess.h`	first the **data**: everything else uses it
2	the syscall number	`syscall-definitions.h`	user space and kernel must agree on the number
3	the `case`	`Syscall.cpp`	the kernel's entry point
4	the libc function	`unistd.c`	the program's entry point
5	`sbrk`: move the break	`UserProcess.cpp`	the syscall logic
6	`mapBrkPage`: map one page	`UserProcess.cpp`	the page fault logic
7	the `if` in the handler	`PageFaultHandler.cpp`	connects page faults to step 6



1 The data

`common/include/kernel/UserProcess.h`

1a: two constants, next to the other `#defines` (after line 19)

```
#define BRK_START 0x0006000000000000ULL
#define BRK_MAX (4096 * PAGE_SIZE)
```

`BRK_START`: where the heap begins.

- The kernel has to know where the heap is, so it can recognise "this page fault is a heap access".
- `0x600000000000` is far away from the code (at the bottom, around `0x400000`) and the stacks (at the top, near `0x7FFF...`), so the heap can never collide with them.
- `ULL` = *unsigned long long*. The number is bigger than 32 bits; without `ULL` the compiler might treat it as a too-small type.

`BRK_MAX`: how big the heap may become at most.

- The card says: at most 4096 pages above the start. 4096×4096 bytes = 16 MiB.
- Why a maximum? Without one, a program could move its break into the stack region or up to the kernel, and the kernel could not recognise heap addresses clearly.
- **The brackets:** a `#define` is pure text replacement. Without them, `x / BRK_MAX` would become `x / 4096 * PAGE_SIZE` – wrong. Rule: put the whole replacement in brackets.

1b: two method declarations, in the `public:` part (e.g. after `cancelThread`, line 172)

```
bool mapBrkPage(size_t vpn);
size_t sbrk(ssize_t increment);
```

Why in `UserProcess`?

The break belongs to **one process** and is shared by all its threads, so the methods that use it belong to that process too. They also need `arch_memory_` and `thread_lock_`, which are private members of `UserProcess` – only its own methods can access them.

`mapBrkPage(size_t vpn)`

- Gets a **page number** (not an address), because page tables work with page numbers.
- Returns `bool`: `true` = the access was allowed and the page is there; `false` = it was above the break.

`sbrk(ssize_t increment)`

- `ssize_t` (with an s = **signed**), because the increment can be **negative** (shrinking). `size_t` cannot hold negative numbers: −4096 would become a huge positive number.
- Returns `size_t`: the old break, which is an address – or `(size_t) -1` for an error.

1c: the break itself, in the `private:` part at the very bottom (after `threads_`, line 282)

```
size_t brk_ = BRK_START;
```

- **The most important variable of the task:** where the heap currently ends.
- `= BRK_START`: at the beginning the heap is empty, so its end equals its start. Writing the start value directly here means you do not have to change the long constructor.
- **The trailing `_`:** the naming style of the file for members (`threads_`, `running_threads_`).
- **Protected by `thread_lock_`:** whoever reads or writes `brk_` must hold that lock. The compiler does not enforce this – it is a rule you keep yourself.

Done when

The three parts are in `UserProcess.h`. It does not have to compile yet – the methods do not exist yet.

2 The syscall number `common/include/kernel/syscall-definitions.h`

```
#define sc_sbrk 1700
```

- A syscall is called **by number**: the program puts `1700` into a register and executes the `syscall` instruction; the kernel looks at the number to know what is wanted.
- This one file is included by **both** sides – the kernel (`Syscall.cpp`) and the libc (`unistd.c`) – so both always agree on the number.
- Put it in numeric order next to the others (e.g. after `sc_pthread_detach`).

Done when

One line added.

3 The case `common/source/kernel/Syscall.cpp`

In `Syscall::syscallException`, inside the big `switch`, e.g. after `case sc_execv` (line 92):

```
case sc_sbrk:
  return_value = currentThread->getProcess()->sbrk((ssize_t) arg1);
  break;
```

- `syscallException` is **the** entry point of every syscall: the `switch` picks the function by the number from step 2.
- `arg1` is the first argument the program passed – the increment. It arrives as `size_t` (registers are just 64 bits, without a sign). The cast `(ssize_t)` reads it as signed again, so -4096 stays -4096.
- `currentThread->getProcess()`: the process of the thread that made the syscall – its break is the one to move.
- `return_value =`: whatever you assign here ends up in the program's `rax` register – the return value of `sbrk` in user space. Forget it, and the program never sees the old break.
- No `Syscall::sbrk` function and nothing in `Syscall.h`: the work goes straight to `UserProcess::sbrk`, because the break belongs to the process.

Done when

Three lines in the `switch`.

4 The libc function

`userspace/libc/src/unistd.c`

The stub already exists (line 37) and returns `(void*) -1`. Replace its body:

```
void* sbrk(intptr_t increment)
{
  return (void*) __syscall(sc_sbrk, (size_t) increment, 0x00, 0x00, 0x00, 0x00);
}
```

- This is what the program calls. It does nothing itself – it only **enters the kernel** with the number `sc_sbrk` and the increment as `arg1`.
- `__syscall` always takes the number plus 5 arguments; the unused ones are `0`.
- `(size_t) increment`: the registers carry plain 64-bit values; step 3 turns it back into a signed number.
- `(void*)` around the result: the kernel returns the old break as a number; for the program it is an **address**. On an error the kernel returns `(size_t) -1`, and `(void*) -1` is exactly what POSIX `sbrk` returns on an error – no extra `if` needed.
- **Do not change the signature** (the comment above the stub says so): `sbrk` is already declared in `unistd.h`, and the test uses that declaration. `unistd.c` already includes `syscall-definitions.h`.

Done when

The stub calls the kernel. Now steps 1–4 compile together except for the two missing methods.

5 `sbrk`: move the break

`common/source/kernel/UserProcess.cpp`

At the end of the file:

```
size_t UserProcess::sbrk(ssize_t increment)
{
  ScopeLock lock(thread_lock_);
  size_t old_brk = brk_;
  size_t new_brk = old_brk + increment;
  if (new_brk < BRK_START || new_brk > BRK_START + BRK_MAX)
    return (size_t) -1;
  if (increment < 0)
  {
    size_t first_vpn = (new_brk + PAGE_SIZE - 1) / PAGE_SIZE;
    size_t end_vpn = (old_brk + PAGE_SIZE - 1) / PAGE_SIZE;
    for (size_t vpn = first_vpn; vpn < end_vpn; ++vpn)
    {
      if (arch_memory_->resolveMapping(vpn).page_size != 0)
        arch_memory_->unmapPage(vpn);
    }
    ArchMemory::flushTlb();
  }
  brk_ = new_brk;
  return old_brk;
}
```

Line 3 – the lock. `brk_` and the page tables are shared by all threads of the process. `ScopeLock` takes `thread_lock_` here and releases it automatically at *every* `return` – you cannot forget to unlock. (If the compiler does not know `ScopeLock`, add `#include "ScopeLock.h"` at the top.)

Line 4 – remember the old break. `sbrk` returns the **old** break: it is exactly where the new memory starts, so `p = sbrk(100)` gives the program `p[0] ... p[99]`. This is how `malloc` gets memory from the kernel.

Line 5 – the new break. Old plus increment; with a negative increment it moves down.

Lines 6–7 – the boundaries. Below the start or more than 4096 pages above it: error, and **nothing** changes (the test checks that a failed `sbrk` leaves the break where it was). Example: `sbrk(-20000)` when the break is at start + 10000 would give start − 10000 – below the start. The same check also catches *wrap-around*: a huge negative increment would make `new_brk` wrap past 0 to a huge number, which is `> BRK_START + BRK_MAX`.

Line 8 – only when shrinking. Growing needs no work here: the pages are mapped later, on first touch, by the page fault handler. That is **demand paging** – RAM only for pages that are used.

Lines 10–11 – which pages go away. `(x + PAGE_SIZE - 1) / PAGE_SIZE` is **rounding up** to a page number. The break can be in the *middle* of a page (`sbrk(100)`); that page is still partly in use and must **stay**. So the first page to remove is the one *after* the new break's page (rounded up), and the loop ends at the old break's page (rounded up, exclusive). Example: old break = start + 10000 (inside page 2), new = start + 100 (inside page 0) ⇒ remove pages 1 and 2, keep page 0.

Line 14 – mapped at all? Demand paging: a page below the old break was maybe **never touched**, so it has no RAM. `resolveMapping` (from A1-09 vtop) tells you; `page_size == 0` means nothing is mapped there.

Line 15 – unmap. `unmapPage` removes the page-table entry **and** frees the physical page. It **panics** (assert) on a page that is not mapped – that is why line 14 checks first.

Line 17 – flush the TLB. The TLB is the CPU's cache of "page → RAM". After an unmap it may still remember the old entry, and the program could keep writing into RAM that is already free. `flushTlb` makes the CPU forget, so the next access really faults.

Line 19 – store. No rounding: the break can be at any byte (the test checks `sbrk(0) == start + 10000`). Rounding is only used to find the affected pages.

Line 20 – return the old break.

Trap

The difference to A0-03: heaplimit stores a *size* rounded to pages; sbrk stores an *address* exactly. Round only when you turn addresses into page numbers.

Done when

`sbrk` is below the other `UserProcess` methods. The heap can move, but touching it still crashes – steps 6 and 7 fix that.

6 mapBrkPage: map one page

`common/source/kernel/UserProcess.cpp`

```
1  bool UserProcess::mapBrkPage(size_t vpn)
2  {
3    size_t ppn = PageManager::instance()->allocPPN();
4    thread_lock_.acquire();
5    bool below = vpn * PAGE_SIZE < brk_;
6    bool mapped = false;
7    if (below)
```

```
8       mapped = arch_memory_->mapPage(vpn, ppn, 1);
9     thread_lock_.release();
10    if (!mapped)
11      PageManager::instance()->freePPN(ppn);
12    return below;
13  }
```

Line 3 – a fresh physical page. `allocPPN()` (no argument – the parameter is a page *size*, not a page number!) finds a free page of RAM and **fills it with zeros**. Zeros matter: otherwise the program could read old data of another program – a security hole. It is done *before* the lock: `allocPPN` has its own lock and searches the whole RAM, so `thread_lock_` is held only briefly.

Line 4 – lock. Check and map must be **one** critical section: otherwise another thread could call `sbrk` and shrink the break between "is it below?" and "map it" – and a page would end up mapped above the break.

Line 5 – below the break? `vpn * PAGE_SIZE` is the **start address** of the page. If the break is in the middle of a page, that page's start is below it ⇒ allowed (the program may use the bytes below the break on that page).

Line 8 – map. `mapPage(vpn, ppn, 1)` writes "virtual page `vpn` → physical page `ppn`" into the page table; `1` = the user may access it. It returns `false` if the page is **already mapped** – another thread of the process touched the same page at the same time and was faster.

Line 9 – unlock. Explicit `acquire`/`release` instead of `ScopeLock`, because lines 10–11 must run *without* the lock.

Lines 10–11 – give the page back. Not mapped (above the break, or the other thread was faster) ⇒ the physical page is unused; without `freePPN` it would be lost forever (a leak).

Line 12 – the answer. `below`, not `mapped`: if the other thread was faster, the access is still allowed – the page exists now. Only "above the break" is `false`.

Done when

Both `UserProcess` methods exist; the kernel compiles.

7 The `if` in the handler

`common/source/mm/PageFaultHandler.cpp`

In `handlePageFault`, as the **first** thing inside `if (checkPageFaultIsValid(address, user, present, switch_to_us)) {` (line 73), before anything else in that block:

```
1   if (address >= BRK_START && address < BRK_START + BRK_MAX)
2   {
3     if (!currentThread->getProcess()->mapBrkPage(address / PAGE_SIZE))
4     {
5       debug(PAGEFAULT, "access above the program break\n");
6       Syscall::exit(9997);
7     }
8     return;
9   }
```

Where `address` comes from. It is a parameter of `handlePageFault`: the address the program tried to access. The CPU stores it in register `cr2` on a page fault; the interrupt entry passes it on.

Line 1 – heap region? Start **included** (`>=`), end **excluded** (`<`). The check uses the **maximum** region, not the current break: an access above the break is still a heap access – a forbidden one, which you must recognise to kill the process. Without this check it would fall through to `loadPage`, which finds no section of the binary there (exit 666).

Line 3 – ask `mapBrkPage`. With the **page number** `address / PAGE_SIZE`. It takes `thread_lock_` itself – that is why this `if` must come *before* the handler's own `getThreadLock().acquire()` further down. Taking a `Mutex` twice is a deadlock.

Lines 5–6 – above the break. End the process. `mapBrkPage` has already released the lock – **never** call `exit` while holding a lock: the thread ends, the lock is never released, and every other thread of the process hangs.

Line 8 – `return`. The fault is handled; the stack code and `loadPage` below must not run. When the handler returns, the CPU **repeats the instruction** that faulted – now the page is mapped and it works. The program never notices.

Done when

`task test` shows **7 of 7 passed**, and the last access ends the process with `exit_code: 9997`.

The test – what each check means

Check	Which step makes it pass
sbrk returns the old break	step 5, line 4 + 20; step 3 `return_value`
break moved by 10000 bytes	step 5, line 19 (no rounding)
the 10000 bytes are usable	steps 6 + 7: three pages mapped on first touch
second allocation follows the first	the break is exact; the old break is where new memory starts
shrinking below the start fails	step 5, line 6 (wrap-around too)
a failed sbrk changes nothing	step 5, line 7 returns before anything changes
bytes below the break survive shrinking	step 5, lines 10–11: page 0 is kept (round up)
last access kills (exit 9997)	step 7, lines 5–6

Say it in three sentences

Remember

- **`sbrk`** moves the end of the heap; when it moves down, it unmaps the pages that are now completely above it; it returns the old end.
- **`mapBrkPage`**: below the end? Then get a zeroed RAM page and map it.
- **the handler**: heap address? Then ask `mapBrkPage`; if it says no, end the program.

Questions a tutor might ask

Question	Short answer
Why does `sbrk` return the *old* break?	it is where the new memory starts; `malloc` uses it as the pointer
Why not map everything in `sbrk` right away?	RAM would be wasted on pages that are never touched (demand paging)
Why are new pages zeroed?	otherwise a program could read another program's old data
Why round *up* when shrinking?	a page still partly below the break is in use and must stay
Why `resolveMapping` before `unmapPage`?	not every page was touched; `unmapPage` panics on an unmapped page
Why `flushTlb`?	the CPU may still cache the old translation and write into freed RAM
Why `ssize_t` for the increment?	it can be negative
Why is `allocPPN` outside the lock?	it is slow and has its own lock; `thread_lock_` stays short
Two threads touch the same new page at once?	both fault; the second `mapPage` returns false → `freePPN`, still allowed
What about `fork`?	the child copies the pages and should copy `brk_` too
What do real systems use?	`mmap` for big allocations; `sbrk` for the classic heap